Programar em C/Ponteiros

< Programar em C

Poderíamos escrever um livro inteiro sobre ponteiros, pois o conteúdo é demasiadamente extenso. Por esse motivo este assunto foi dividido em **básico**, **intermediário** e **avançado**, assim o leitor poderá fazer seus estudos conforme suas necessidades.

É recomendável para quem está vendo programação pela primeira vez aqui que não se preocupe com o avançado sobre ponteiros por enquanto.

Índice

Básico
- O que é um ponteiro?
- Declarando e acessando ponteiros
- Ponteiro e NULL
- Mais operações com ponteiros

Intermediário
- Ponteiro de estrutura
- Ponteiros como parâmetros de funções
- Ponteiros e vetores
- Indexação estranha de ponteiros
- Comparando endereços

Avançado
- Ponteiros para ponteiros
- Passando vetores como argumentos de funções
- Ponteiros para funções

Básico

O que é um ponteiro?

Um ponteiro é simplesmente uma variável que armazena o endereço de outra variável.

Um exemplo : O que é o ponteiro de um relógio? É o que aponta para as horas, minutos ou segundos. Um ponteiro aponta para algo. Em programação, temos as variáveis armazenadas na memória, e um ponteiro aponta para um endereço de memória.

Imagine as variáveis como documentos, a memória do computador como pastas para guardar os documentos, e o ponteiro como atalhos para as pastas.

Não se desespere caso não consiga entender num primeiro momento, o conceito fica mais claro com a prática.

Declarando e acessando ponteiros

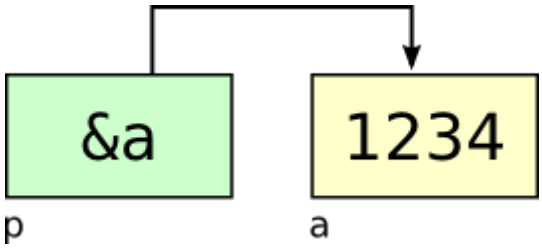
Um ponteiro, como qualquer variável, deve ter um tipo, que é o tipo da variável para a qual ele aponta. Para declarar um ponteiro, especificamos *o tipo da variável para a qual ele aponta* e seu nome **precedido por asterisco**:

```
int  ponteiro  ;        /* declara uma variável comum do tipo inteiro */
int *ponteiro  ;        /* declara um ponteiro para um inteiro */
```

Tome cuidado ao declarar vários ponteiros em uma linha, pois o asterisco deve vir **antes de cada nome de variável**. Note os três exemplos:

```
int  p,  q,  r; // estamos a declarar três variáveis comuns
int *p,  q,  r; // cuidado! apenas p será um ponteiro!
int *p, *q, *r; // agora sim temos três ponteiros
```

Para acessar o endereço de uma variável, utilizamos o operador **&** (E comercial), chamado "operador de referência" ou "operador de endereço". Como o nome sugere, ele retorna o endereço na memória de seu operando. Ele é *unário* e deve ser escrito antes do seu operando. Por exemplo, se uma variável `nome` foi guardada no endereço de memória 1000, a expressão `&nome` valerá 1000.



Esquema de um ponteiro

Com isso, fica claro o esquema ao lado: a variável *a* contém o valor 1234 e o ponteiro *p* contem o endereço de *a* (`&a`).

Para atribuir um valor **ao ponteiro**, usamos apenas seu nome de variável. Esse valor deve ser um endereço de memória, portanto obtido com o operador `&`:

```
int  a;
int *p;
p = &a;
```

Claro que também podemos inicializar um ponteiro:

```
int *p = &a;
```

Nos dois casos, o ponteiro *p* irá apontar para a variável *a*.

Mas, como o ponteiro contém um endereço, podemos também atribuir um valor à variável guardada nesse endereço, ou seja, **à variável apontada pelo ponteiro**. Para isso, usamos o operador ***** (asterisco), que basicamente significa "o valor apontado por".
Ex:

```
int  a ;
int *p = &a ;
*p = 20 ;
```

Para ver o resultado :

```
printf (" a :%i\n",  a);
printf ("*p :%i\n", *p);
```

> **Cuidado!** Você **nunca** deve usar um ponteiro sem antes inicializá-lo; esse é um erro comum. Inicialmente, um ponteiro pode apontar para qualquer lugar da memória do computador. Ou seja, ao tentar ler ou gravar o valor apontado por ele, você estará manipulando um lugar desconhecido na memória!

```
int *p;
*p = 9;
```

Nesse exemplo, estamos a manipular um lugar desconhecido da memória! Se você tentar compilar esse código, o compilador deverá dar uma mensagem de aviso; durante a execução, provavelmente ocorrerá uma falha de segmentação (erro que ocorre quando um programa tenta acessar a memória alheia).

Um exemplo mais elaborado:

```
#include <stdio.h>

int main()
{
   int  i = 10 ;
   int *p ;
   p = &i ;
   *p =  5 ;

   printf ("%d\t%d\t%p\n", i, *p, p);
   return 0;
}
```

Primeiramente declaramos a variável *i*, com valor 10, e o ponteiro *p*, que apontará para o endereço de *i*. Depois, guardamos o valor 5 no endereço apontado por *p*. Se você executar esse exemplo, verá algo parecido com:

```
5     5      0022FF74
```

É claro que os valores de `i` e de `*p` serão iguais, já que *p* aponta para *i*. O terceiro valor é o endereço de memória onde está *i* (e, consequentemente, é o próprio valor de *p*), e será diferente em cada sistema.

Cuidado! Os operadores unários `&` e `*` não podem ser confundidos com os operadores binários **AND** bit a bit e **multiplicação**, respectivamente.

Ponteiro e NULL

Uma falha de segmentação ou em inglês (segmentation fault) ocorre quando um programa tenta acessar um endereço na memória que está reservado ou que não existe.Nos sistemas Unix quando acontece este tipo de erro o sinal SIGSEGV é enviado ao programa indicando uma falha de segmentação.

Aqui o ponteiro contem null, definido com o endereço (0x00000000) que causa uma falha de segmentação .

```
/*Endereço invalido*/
#define null ( (char*) 0 )

int main(void){

  int  a = 5;
  int *p = null;
      *p = a;
}
```

Esse programa termina anormalmente. Você esta tentando colocar o valor 5 em um endereço inválido.

Para que isso não aconteça o ponteiro deve ser inicializado com um endereço valido. Exemplo :

```
#include <stdio.h>
#include <errno.h>
#include <stddef.h>

int main(void){

  int  a = 5;
  int *p = NULL;
       p = &a;

  /* A operação não é permitida */
  if(p == NULL) return -EPERM ;

     else{

          printf("Endereço a disposição:%p\n", p );
          *p = a; /* Pode colocar 5 */
    }
}
```

NULL está definido dentro do cabeçalho stddef.h . Aqui você não espera que o programa acabe com algum tipo de mágica, se NULL é igual ao valor do ponteiro isso significa que não foi encontrado nem um endereço acessível, então você para. Caso contrario você estará executando uma operação que não é permitida. Ou colocar 5 em (0x00000000) .

Mais operações com ponteiros

Suponhamos dois ponteiros inicializados *p1* e *p2*. Podemos fazer dois tipos de atribuição entre eles:

```
p1 = p2;
```

Esse primeiro exemplo fará com que *p1* aponte para o mesmo lugar que *p2*. Ou seja, usar *p1* será equivalente a usar *p2* após essa atribuição.

```
*p1 = *p2;
```

Nesse segundo caso, estamos a igualar **os valores apontados** pelos dois ponteiros: alteraremos o valor apontado por *p1* para o valor apontado por *p2*.

Agora vamos dar mais alguns exemplos com o ponteiro *p*:

```
p++;
```

Aqui estamos a incrementar *o ponteiro*. Quando incrementamos um ponteiro ele passa a apontar para o **próximo valor do mesmo tipo** em relação ao valor para o qual o ponteiro aponta. Isto é, se temos um ponteiro para um inteiro e o incrementamos, ele passa a apontar para o próximo inteiro. **Note que o incremento não ocorre byte-a-byte!**

```
(*p)++;
```

Aqui, colocamos `*p` entre parênteses para especificar que queremos alterar o valor apontado por *p*. Ou seja, aqui iremos incrementar o conteúdo da variável apontada pelo ponteiro p.

```
*p++
```

Neste caso, o efeito não é tão claro quanto nos outros exemplos. A *precedência* do operador ++ sobre o operador * faz com que a expressão seja equivalente a `(*p)++`. O valor atual de *p* é retornado ao operador *, e o valor de `p` é incrementado. Ou seja, obtemos o valor atual do ponteiro e já o fazemos apontar para o próximo valor.

```
x = *(p + 15);
```

Esta linha atribui a uma variável *x* o conteúdo do décimo-quinto inteiro adiante daquele apontado por *p*. Por exemplo, suponhamos que tivéssemos uma série de variáveis *i0*, *i1*, *i2*, ... *i15* e que *p* apontasse para *i0*. Nossa variável *x* receberia o valor de *i15*.

Tente acompanhar este exemplo dos dois tipos de atribuição de ponteiros:

```
int *a, *b, c = 4, d = 2;
a = &c;        // a apontará para c
b = &d;        // b apontará para d
*b = 8;        // altero o valor existente na variavel d
*a = *b;       // copio o valor de d (apontado por b)
                 // para c (apontado por a)
*a = 1;        // altero o valor da variável c
b = a;         // b aponta para o mesmo lugar que a,
                 // ou seja, para c
*b = 0;        // altero o valor de c
```

Intermediário

Ponteiro de estrutura

Para começar e deixar mais claro definimos uma estrutura simples com dois campos.

```
 struct {
          int i;
          double f;
        } minha_estrutura;
```

O passo seguinte é definir um ponteiro para essa estrutura.

```
struct minha_estrutura *p_minha_estrutura;
```

A partir do ponteiro podemos ter acesso a um campo da estrutura usando um seletor "->" (uma flecha).

```
p_minha_estrutura-> i = 1;
p_minha_estrutura-> f = 1.2;
```

O mesmo resultado pode ser obtido da seguinte forma.

```
(*p_minha_estrutura).i = 1;
(*p_minha_estrutura).f = 1.2;
```

O operador cast também e bastante utilizado para estruturar áreas de estoque temporários (buffer). Os tipos dentro da estrutura devem ser o mesmo do arranjo para evitar problemas de alinhamento.

A seguir um pequeno exemplo:

```
#include <stdio.h>
typedef struct estruturar{
```

```
   char a ;
   char b ;
} estruturar;

int main()

{
   char buffer[2] = {17, 4};
   estruturar *p;
   p = (struct estruturar*) &buffer;

   printf("a: %i b: %i", p->a,p->b);
   // getchar(); /* Se o ambiente for windows, descomente o começo da linha. */
    return 0;

}
```

Ponteiros como parâmetros de funções

Comecemos por uma situação-problema: eu tenho 2 variáveis e quero trocar o valor delas. Vamos começar com um algoritmo simples, dentro da função main():

```
#include <stdio.h>

int main()
{
   int a = 5, b = 10, temp;
   printf ("%d %d\n", a, b);

   temp = a;
   a = b;
   b = temp;

   printf ("%d %d\n", a, b);
   return 0;
}
```

Esse exemplo funcionará exatamente como esperado: primeiramente ele imprimirá "5 10" e depois ele imprimirá "10 5". Mas e se quisermos trocar várias vezes o valor de duas variáveis? É muito mais conveniente criar uma função que faça isso. Vamos fazer uma tentativa de implementação da função swap (troca, em inglês):

```
#include <stdio.h>

void swap(int i, int j)
{
   int temp;
   temp = i;
   i = j;
   j = temp;
}

int main()
{
   int a, b;
   a = 5;
   b = 10;
   printf ("%d %d\n", a, b);
   swap (a, b);
   printf ("%d %d\n", a, b);
   return 0;
}
```

No entanto, o que queremos não irá acontecer. Você verá que o programa imprime duas vezes "5 10". Por que isso acontece? Lembre-se do escopo das variáveis: as variáveis *a* e *b* são locais à função main(), e quando as passamos como argumentos para swap(), seus valores são copiados e passam a ser chamados de *i* e *j*; a troca ocorre entre *i* e *j*, de modo que quando voltamos à função main() nada mudou.

Então como poderíamos fazer isso? Como são retornados dois valores, não podemos usar o valor de retorno de uma função. Mas existe uma alternativa: os **ponteiros**!

```
#include <stdio.h>

void swap (int *i, int *j)
{
   int temp;
   temp = *i;
   *i = *j;
   *j = temp;
}

int main ()
{
   int a, b;
   a = 5;
   b = 10;
   printf ("\n\nEles valem %d, %d\n", a, b);
   swap (&a, &b);
   printf ("\n\nEles agora valem %d, %d\n", a, b);
   return 0;
}
```

Neste exemplo, definimos a função swap() como uma função que toma como argumentos dois ponteiros para inteiros; a função faz a troca *entre os valores apontados pelos ponteiros*. Já na função main(), passamos *os endereços das variáveis* para a função swap(), de modo que a função swap() possa modificar variáveis locais de outra função. O único possível inconveniente é que, quando usarmos a função, teremos de lembrar de colocar um & na frente das variáveis que estivermos passando para a função.

Se você pensar bem, já vimos uma função em que passamos os argumentos precedidos de &: é a função *scanf()*! Por que fazemos isso? É simples: chamamos a função scanf() para que ela ponha nas nossas variáveis valores digitados pelo usuário. Ora, essas variáveis são locais, e portanto só podem ser alteradas por outras funções através de ponteiros!

Quando uma função recebe como parâmetros os endereços e não os valores das variáveis, dizemos que estamos a fazer uma **chamada por referência**; é o caso desse último exemplo. Quando passamos diretamente os valores das variáveis para uma função, dizemos que é uma **chamada por valor**; foi o caso do segundo exemplo. Veja mais um exemplo abaixo:

```
// passagem_valor_referencia.c

#include<stdio.h>

int cubo_valor( int );

int cubo_referencia( int * );

int main(){
  int number = 5;

  printf("\nO valor original eh: %d", number );

  number = cubo_valor( number );
  printf("\nO novo valor de number eh: %d", number);

  printf("\n---------------");

  number = 5;
  printf("\nO valor original eh: %d", number );

  cubo_referencia( &number );

  printf("\nO novo valor de number eh: %d", number);

  return 0;
```

```
}

int cubo_valor( int a){
  return a * a * a;
}

int cubo_referencia( int *aPtr ){
  *aPtr = *aPtr  *  *aPtr  *  *aPtr;
  return *aPtr;
}
```

Ponteiros e vetores

Em C, os elementos de um vetor são sempre guardados sequencialmente, a uma distância fixa um do outro. Com isso, é possível facilmente passar de um elemento a outro, percorrendo sempre uma mesma distância para frente ou para trás na memória. Dessa maneira, podemos usar ponteiros e a aritmética de ponteiros para percorrer vetores. Na verdade, vetores *são* ponteiros — um uso particular dos ponteiros. Acompanhe o exemplo a seguir.

```
#include <stdio.h>

int main ()
{
   int i;
   int vetorTeste[3] = {4, 7, 1};
   int *ptr = vetorTeste;

   printf("%p\n", vetorTeste);
   printf("%p\n", ptr);
   printf("%p\n", &ptr);

   for (i = 0; i < 3; i++)
   {
      printf("O endereço do índice %d do vetor é %p\n", i, &ptr[i]);
      printf("O valor do índice %d do vetor é %d\n", i, ptr[i]);
   }
   return 0;
}
```

Começamos declarando um vetor com três elementos; depois, criamos um ponteiro para esse vetor. Mas repare que **não colocamos o operador de endereço** em vetorTeste; fazemos isso porque um vetor já representa um endereço, como você pode verificar pelo resultado da primeira chamada a printf().

Como você já viu anteriormente neste capítulo, podemos usar a sintaxe `*(ptr + 1)` para acessar o inteiro seguinte ao apontado pelo ponteiro *ptr*. Mas, se o ponteiro aponta para o vetor, o próximo inteiro na memória será o próximo elemento do vetor! De fato, em C as duas formas `*(ptr + n)` e `ptr[n]` são equivalentes.

Não é necessário criar um ponteiro para usar essa sintaxe; como já vimos, o vetor em si já é um ponteiro, de modo que qualquer operação com *ptr* será feita igualmente com *vetorTeste*. Todas as formas abaixo de acessar o segundo elemento do vetor são equivalentes:

```
vetorTeste[1];
*(vetorTeste + 1);
ptr[1];
*(ptr + 1)
```

Veja mais este exemplo:

```
#include <stdio.h>

int main()
{
   int numbers[5];
```

```
    int *p;
    int n;

    p = numbers;
    *p = 10;
    p++;
    *p = 20;
    p = &numbers[2];
    *p = 30;
    p = numbers + 3;
    *p = 40;
    p = numbers;
    *(p + 4) = 50;

    for (n = 0; n < 5; n++)
       cout << numbers[n] << ", ";
    return 0;
}
```

Ele resume as várias formas de acessar elementos de um vetor usando ponteiros.

Indexação estranha de ponteiros

o C permite fazer um tipo indexação de um vetor quando uma variável controla seu índice. O seguinte código é válido e funciona: Observe a indexação **vetor[i]**.

```
#include <stdio.h>
int main ()
{
   int i;
   int vetor[10];

   for (i = 0; i < 10; i++) {
      printf ("Digite um valor para a posicao %d do vetor: ", i + 1);
      scanf ("%d", &vetor[i]); //isso é equivalente a fazer *(x + i)
   }

   for (i = 0; i < 10; i++)
      printf ("%d\n", vetor[i]);

   return (0);
}
```

Essa indexação, apesar de estranha, funciona corretamente e sem aviso na compilação. Ela é prática, mas, para os iniciantes, pode parecer complicada. É só treinar para entender.

Comparando endereços

Como os endereços são números, eles também podem ser comparados entre si. Veja o exemplo a seguir, com efeito equivalente ao primeiro exemplo da seção anterior:

```
#include <stdio.h>

int main()
{
   int vetorTeste[3] = {4, 7, 1};
   int *ptr = vetorTeste;
   int i = 0;

   while (ptr <= &vetorTeste[2])
   {
      printf("O endereço do índice %d do vetor é %p\n", i, ptr);
      printf("O valor do índice %d do vetor é %d\n", i, *ptr);
      ptr++;
      i++;
   }
   return 0;
}
```

Esse programa incrementa o ponteiro enquanto esse endereço for igual (ou menor) ao endereço do último elemento do vetor (lembre-se que os índices do vetor são 0, 1 e 2).

Avançado

Este módulo precisa ser revisado por alguém que conheça o assunto (discuta).

Este módulo tem a seguinte tarefa pendente: mover o que é de C++ para o livro de C++

Ponteiros para ponteiros

Note que um ponteiro é uma variável como outra qualquer, e por isso também ocupa espaço em memória. Para obtermos o endereço que um ponteiro ocupa em memória, usamos o operador `&`, assim como fazemos nas variáveis comuns.

Mas e se estivéssemos interessados em guardar o endereço de um ponteiro, que tipo de váriavel deveria recebe-lo? A resposta é: um ponteiro, isto é, um ponteiro para outro ponteiro.

Considere a seguinte declaração:

```
int x = 1;
```

Declaramos uma variável chamada `x` com o valor 1.

Como já sabemos, para declarar um ponteiro, deve-se verificar o tipo da variável que ele irá apontar (neste caso `int`) e colocar um asterisco entre o tipo da variável e o nome do ponteiro:

```
int * p_x = &x;
```

Declaramos um ponteiro apontado para `x`.

Agora, para se guardar o endereço de um ponteiro, os mesmos passos devem ser seguidos. Primeiramente verificamos os tipo da variável que será apontada (`int *`) e colocamos um asterisco entre o tipo e nome do ponteiro:

```
int ** p_p_x = &p_x;
```

Declaramos um ponteiro que irá apontar para o ponteiro `p_x`, ou seja, um ponteiro para ponteiro. Note que C não impõe limites para o número de asteriscos em uma variável.

No exemplo a seguir, todos os `printf` irão escrever a mesma coisa na tela.

```
#include <stdio.h>

int main(void)
{
    int x = 1;
    int *p_x = &x;    // p_x aponta para x
    int **p_p_x = &p_x;     // p_p_x aponta para o ponteiro p_x

    printf("%d\n", x);   // Valor da variável
    printf("%d\n", *p_x);    // Valor da variável apontada por p_x
    printf("%d\n", **p_p_x);    // Valor da variável apontada pelo endereço apontado por p_p_x
    return 0;
}
```

Percebe que `**p_p_x` consiste no valor da variável apontada pelo endereço apontado por `p_p_x`.

Uma aplicação de ponteiros para ponteiros está nas strings, já que strings são vetores, que por sua vez são ponteiros. Um vetor de strings seria justamente um ponteiro para um ponteiro.

Este módulo tem a seguinte tarefa pendente: Criar código exemplo sobre **ponteiro para ponteiro**

Passando vetores como argumentos de funções

Os ponteiros podem ser passados como argumentos de funções.

Parâmetro ponteiro passando um array.

```
#include <stdio.h>

void atribuiValores(int[], int);
void mostraValores(int[], int);

int main()
{
   int vetorTeste[3];  // crio um vetor sem atribuir valores
   atribuiValores(vetorTeste, 3);
   mostraValores(vetorTeste, 3);
   return 0;
}

void atribuiValores(int valores[], int num)
{
   for (int i = 0; i < num; i++)
   {
      printf("Insira valor #%d: ", i + 1);
      scanf("%d", &valores[i]);
   }
}

void mostraValores(int valores[], int num)
{
   for (int i = 0; i < num; i++)
   {
      printf("Valor #%d: %d\n", i + 1, valores[i]);
   }
}
```

Repare que passamos dois parâmetros para as funções:

1. O "nome" do vetor, que representa o seu endereço na memória. (Temos 3 maneiras para passar o endereço do vetor: diretamente pelo seu "nome", via um ponteiro ou pelo endereço do primeiro elemento.)
2. Uma constante, que representa o número de elementos do vetor. Isso é importante pois o C não guarda informações sobre o tamanho dos vetores; você não deve tentar alterar ou acessar valores que não pertencem ao vetor.

É claro que devemos passar o endereço do vetor (por "referência"), pois os seus valores são alterados pela função atribuiValores. De nada adiantaria passar o vetor por valor, pois o valor só seria alterado localmente na função (como já vimos no caso de troca do valor de duas variáveis).

Por causa dessa equivalência entre vetores e ponteiros, podemos fazer uma pequena alteração no protótipo (tanto na declaração quanto na definição) das funções atribuiValores e mostraValores, sem precisar alterar o código interno dessas funções ou a chamada a elas dentro da função main ? trocando

```
void atribuiValores(int[], int);
void mostraValores(int[], int);
```

por

```
void atribuiValores(int*, int);
void mostraValores(int*, int);
```

Para o compilador, você não fez mudança alguma, justamente por conta dessa equivalência. Em ambos os casos, foi passado o endereço do vetor para as funções.

Ponteiros para funções

Os ponteiros para funções servem, geralmente, para passar uma função como argumento de uma outra função. Neste exemplo

```
#include <stdio.h>

int soma(int a, int b)
{
   return (a + b);
}

int operacao(int x, int y, int (*func)(int,int))
{
   int g;
   g = (*func)(x, y);
   return (g);
}

int main ()
{
   int m;
   m = operacao(7, 5, soma);
   printf("%d\n", m);
   return 0;
}
```

Veja que criamos uma função que retorna a soma dos dois inteiros a ela fornecidos; no entanto, ela não é chamada diretamente. Ela é chamada pela função *operacao*, através de um ponteiro. A função *main* passa a função *soma* como argumento para *operacao*, e a função *operacao* chama essa função que lhe foi dada como argumento.

Note bem o terceiro argumento da função *operacao*: ele é um **ponteiro para uma função**. Nesse caso, ele foi declarado como um ponteiro para uma função que toma dois inteiros como argumentos e retorna outro inteiro. O * indica que estamos declarando um ponteiro, e não uma função. **Os parênteses em torno de `*func` são essenciais**, pois sem eles o compilador entenderia o argumento como *uma função que retorna um ponteiro para um inteiro*.

A forma geral para declarar um ponteiro para uma função é:

```
tipo_retorno (*nome_do_ponteiro)(lista de argumentos)
```

Para chamar a função apontada pelo ponteiro, há duas sintaxes. A sintaxe original é

```
(*nome_do_ponteiro)(argumentos);
```

Se *ptr* é um ponteiro para uma função, faz bastante sentido que a função em si seja chamada por `*ptr`. No entanto, a sintaxe mais moderna permite que ponteiros para funções sejam chamados exatamente da mesma maneira que funções:

```
nome_do_ponteiro(argumentos);
```

Por fim, para inicializar um ponteiro para função, não precisamos usar o operador de endereço (ele já está implícito). Por isso, quando chamamos a função *operacao*, não precisamos escrever `&soma`.

Veja mais um exemplo — na verdade, uma extensão do exemplo anterior:

```
#include <stdio.h>

int soma(int a, int b)
{
   return (a+b);
}

int subtracao(int a, int b)
{
   return (a-b);
}

int (*menos)(int, int) = subtracao;

int operacao(int x, int y, int (*func)(int,int))
{
   int g;
   g = func(x, y);
   return (g);
}

int main()
{
   int m, n;
   m = operacao(7, 5, soma);
   n = operacao(20, m, menos);
   printf("%d\n", n);
   return 0;
}
```

Aqui, criamos mais uma função, *subtracao*, além de criar um outro ponteiro para ela (uma espécie de "atalho"), *menos*. Na função *main*, referimo-nos à função de subtração através desse atalho.

Veja também que aqui usamos a sintaxe moderna para a chamada de ponteiros de funções, ao contrário do exemplo anterior.

Obtido em "https://pt.wikibooks.org/w/index.php?title=Programar_em_C/Ponteiros&oldid=447619"

Esta página foi editada pela última vez à(s) 13h23min de 28 de setembro de 2017.

Este texto é disponibilizado nos termos da licença Creative Commons Atribuição-Compartilhamento pela mesma Licença 3.0 Unported; pode estar sujeito a condições adicionais. Consulte as Condições de Uso para mais detalhes.